Chapter 1 | Worked solutions with Python

Tokens, Embeddings, and Neural Language Models

All 12 exercises follow the revised chapter's numbering. First work through each question on your own, then use the explanation and the executable check. The weights are chosen by hand; this notebook does not train a language model. The written PDF contains every explanation and answer. The notebook PDF also includes every code cell and its executed output.

Setup

Run the notebook inside a copy of `transformers-book` with NumPy installed. The setup locates the repository from either its root or this notebook's folder. The exercise statements are in `examples/ch01_language/EXERCISES.md`.

In [1]

```
from pathlib import Path
import sys
import numpy as np

for folder in (Path.cwd(), *Path.cwd().parents):
    if (folder / "examples/ch01_language/model.py").is_file():
        if str(folder) not in sys.path:
            sys.path.insert(0, str(folder))
        break
else:
    raise FileNotFoundError("Open this notebook inside transformers-book.")

from examples.ch01_language.model import (
    BOS, EOS, TOKEN_TO_ID, VOCAB, context_logits, embedding_lookup,
    next_token_pairs, stable_softmax, token_nll, toy_parameters,
)

np.set_printoptions(precision=6, suppress=True)
parameters = toy_parameters()
ids = np.array([TOKEN_TO_ID[w] for w in "the red car".split()])
```

1. Separate the three representations

Start with three tokens: `the`, `red`, `car`. Their IDs are `(4, 12, 7)`. Each ID selects a row of the same embedding table:

Position	Token	ID	Retrieved embedding
First	the	4	(0.2, -0.4, 0.1, 0.5)
Second	red	12	(-0.3, 0.8, 0.2, -0.1)
Third	car	7	(0.6, 0.1, -0.5, 0.3)

The ID is an integer address. Its magnitude is not a measurement of meaning. The embedding is the four-number row stored at that address. The table `E` has shape `(16, 4)` because it contains one row per vocabulary entry. Stack the three selected rows in sentence order and `X` has shape `(3, 4)`. A row of `E` belongs to a token type; a row of `X` belongs to a position in this phrase.

In [2]

```
x = embedding_lookup(parameters["E"], ids)
assert ids.tolist() == [4, 12, 7]
assert parameters["E"].shape == (16, 4)
assert x.shape == (3, 4)
np.testing.assert_array_equal(x, [[0.2, -0.4, 0.1, 0.5],
                                  [-0.3, 0.8, 0.2, -0.1],
                                  [0.6, 0.1, -0.5, 0.3]])
print("IDs:", ids)
print("X:\n", x)
```

Output

```
IDs: [ 4 12  7]
X:
 [[ 0.2 -0.4  0.1  0.5]
 [-0.3  0.8  0.2 -0.1]
 [ 0.6  0.1 -0.5  0.3]]
```

2. Repeat a token

The IDs for `red car red` are `(12, 7, 12)`. Reading those rows in order gives:

```
[-0.3,  0.8,  0.2, -0.1]
[ 0.6,  0.1, -0.5,  0.3]
[-0.3,  0.8,  0.2, -0.1]
```

The first and third rows are equal. Both positions ask for row 12, so both get the same vector. There are still three rows: positions are counted even when tokens repeat. Lookup uses the ID at each position, without consulting its neighbors. A later network can combine the vectors differently in different contexts. The lookup itself has not done that yet.

In [3]

```
repeated_x = embedding_lookup(parameters["E"], [12, 7, 12])
assert repeated_x.shape == (3, 4)
np.testing.assert_array_equal(repeated_x[0], repeated_x[2])
np.testing.assert_array_equal(repeated_x[1], parameters["E"][7])
print(repeated_x)
```

Output

```
[[-0.3  0.8  0.2 -0.1]
 [ 0.6  0.1 -0.5  0.3]
 [-0.3  0.8  0.2 -0.1]]
```

3. Count tokens, not words

The word-level tokenizer gives `the | unhelpful | car`: **3 tokens**. The supplied subword rule gives `the | un | help | ful | car`: **5 tokens**. Replacing one whole word with three pieces adds two positions. The related word `helpful` uses two of the same pieces, illustrating how a vocabulary can reuse units. It is not part of the counted phrase.

A real BPE tokenizer learns merges of adjacent symbols from its training text. Its vocabulary and merge rules determine whether it joins `help` and `ful`, leaves them separate, or chooses another segmentation. The supplied split is a teaching rule, not an output promised by BPE. The small word vocabulary in our Python example does not contain these extra pieces, so the check counts symbolic pieces without assigning them the example model's IDs.

In [4]

```
word_tokens = ["the", "unhelpful", "car"]
teaching_pieces = ["the", "un", "help", "ful", "car"]
assert len(word_tokens) == 3
assert len(teaching_pieces) == 5
print("Word-level count:", len(word_tokens))
print("Teaching subword count:", len(teaching_pieces))
```

Output

```
Word-level count: 3
Teaching subword count: 5
```

4. Read the axes

There are five examples. Each sentence has three IDs; lookup replaces each ID with a four-coordinate vector. The first axis continues to count examples throughout the calculation.

Array	Shape	Meaning of the axes
Sentence IDs	(5, 3)	example, token position
Sentence embeddings	(5, 3, 4)	example, position, coordinate
Context IDs	(5, 2)	example, context position
Context embeddings	(5, 2, 4)	example, context position, coordinate
Joined vectors	(5, 8)	example, joined coordinate
Hidden vectors	(5, 4)	example, hidden coordinate
Logits	(5, 16)	example, vocabulary entry

We take **one context per sentence**, so there are five predictions here. Joining two width-four vectors gives eight coordinates in each example; it does not join different examples. Softmax normalizes the final vocabulary axis separately in each row. Normalizing down the batch axis would compare different prediction questions with one another.

In [5]

```
batch_ids = np.tile(ids, (5, 1))
batch_x = embedding_lookup(parameters["E"], batch_ids)
context_x = embedding_lookup(parameters["E"], batch_ids[:, :2])
joined = context_x.reshape(5, 8)
batch_hidden, batch_logits = context_logits(batch_ids[:, :2], parameters)
assert batch_ids.shape == (5, 3) and batch_x.shape == (5, 3, 4)
assert context_x.shape == (5, 2, 4) and joined.shape == (5, 8)
assert batch_hidden.shape == (5, 4) and batch_logits.shape == (5, 16)
np.testing.assert_allclose(stable_softmax(batch_logits).sum(axis=-1), 1)
print("Sentence IDs / embeddings:", batch_ids.shape, batch_x.shape)
print("Context embeddings / joined:", context_x.shape, joined.shape)
print("Hidden / logits:", batch_hidden.shape, batch_logits.shape)
```

Output

```
Sentence IDs / embeddings: (5, 3) (5, 3, 4)
Context embeddings / joined: (5, 2, 4) (5, 8)
Hidden / logits: (5, 4) (5, 16)
```

5. Normalize a full vocabulary

Exponentiate all scores before adding them. The two leading contributions are `e^2` and `e^1`; each of the other 14 entries contributes `e^0 = 1`. Thus `D = e^2 + e + 14 = 24.1073379274` (rounded).

Entry	Calculation	Probability
car	e^2 / D	0.306506514
red	e / D	0.112757445
Each other entry	1 / D	0.041481146

Before rounding, the normalization is exactly `(e^2 + e + 14) / D = 1`. The zero logits receive positive probability because their exponentials are 1. The largest single probability belongs to `car`, but the other 15 entries together have probability `1 - p(car) = 0.693493486`. That combined event is more probable than `car`. Ranking first does not imply a probability above one half. Adding rounded table entries may leave a tiny rounding discrepancy, so code checks the unrounded values.

In [6]

```
hidden, logits = context_logits([4, 12], parameters)
p = stable_softmax(logits)
denominator = np.exp(2) + np.exp(1) + 14
expected = np.full(16, 1 / denominator)
expected[7], expected[12] = np.exp(2) / denominator, np.exp(1) / denominator
np.testing.assert_allclose(p, expected)
np.testing.assert_allclose(p.sum(), 1)
assert p[7] < 1 - p[7]
print(f"D = {denominator:.10f}")
print(f"p(car) = {p[7]:.9f}; p(red) = {p[12]:.9f}")
print(f"Each other = {p[0]:.9f}; all non-car = {1-p[7]:.9f}")
```

Output

```
D = 24.1073379274
p(car) = 0.306506514; p(red) = 0.112757445
Each other = 0.041481146; all non-car = 0.693493486
```

6. Grade the observed target

For `car`, use the probability in slot 7: `-ln(0.3065065135) = 1.1825162725`. A uniform distribution assigns `1/16` to it and has loss `ln(16) = 2.7725887222`. The worked distribution has the smaller loss because it allocates more probability to the observed target.

If the target is `red`, grade slot 12 instead: the loss is `-ln(0.1127574449) = 2.1825162725`. The target comes from the training text, not from `argmax`. Several continuations may be plausible, but a particular training occurrence supplies one observed continuation. Across many examples, the model can learn to give probability to several alternatives.

Natural logarithms are used throughout. Notice that `red` has a loss exactly one larger than `car` here: their logits differ by one and share the same log-normalizer.

In [7]

```
car_loss = float(token_nll(logits, 7))
red_loss = float(token_nll(logits, 12))
uniform_loss = float(token_nll(np.zeros(16), 7))
np.testing.assert_allclose(car_loss, -np.log(p[7]))
np.testing.assert_allclose(red_loss - car_loss, 1)
np.testing.assert_allclose(uniform_loss, np.log(16))
assert car_loss < uniform_loss
print(f"Car loss: {car_loss:.10f}")
print(f"Uniform loss: {uniform_loss:.10f}")
print(f"Red loss: {red_loss:.10f}")
```

Output

```
Car loss: 1.1825162725
Uniform loss: 2.7725887222
Red loss: 2.1825162725
```

7. Follow the context calculation

For each coordinate, add the first embedding, twice the second embedding, and the bias. ReLU keeps a positive result and replaces a negative result with zero. Coordinates in this table are numbered from 1 for readability.

Coordinate	Calculation for the red	Before ReLU	After ReLU
1	0.2 + 2(-0.3) + 0	-0.4	0
2	-0.4 + 2(0.8) - 0.2	1.0	1.0
3	0.1 + 2(0.2) + 0	0.5	0.5
4	0.5 + 2(-0.1) + 0	0.3	0.3

Reversing the tokens changes which vector receives the factor of two:

Coordinate	Calculation for red the	Before ReLU	After ReLU
1	-0.3 + 2(0.2) + 0	0.1	0.1
2	0.8 + 2(-0.4) - 0.2	-0.2	0
3	0.2 + 2(0.1) + 0	0.4	0.4
4	-0.1 + 2(0.5) + 0	0.9	0.9

The output layer reads only the second hidden coordinate. After `the red`, that coordinate is 1, yielding logits 2 for `car`, 1 for `red`, and 0 for the other 14 entries: the distribution from Exercise 5. After `red the`, that coordinate is 0, so all 16 logits are 0 and every entry receives `1/16`. Other hidden coordinates are positive, but their output weights are zero. The order sensitivity comes from different weights for the two input positions.

In [8]

```
e_the, e_red = parameters["E"][[4, 12]]
forward_pre = e_the + 2 * e_red + parameters["b_context"]
reverse_pre = e_red + 2 * e_the + parameters["b_context"]
forward_h, forward_z = context_logits([4, 12], parameters)
reverse_h, reverse_z = context_logits([12, 4], parameters)
np.testing.assert_allclose(forward_pre, [-0.4, 1, 0.5, 0.3])
np.testing.assert_allclose(reverse_pre, [0.1, -0.2, 0.4, 0.9])
np.testing.assert_allclose(forward_h, [0, 1, 0.5, 0.3])
np.testing.assert_allclose(reverse_h, [0.1, 0, 0.4, 0.9])
np.testing.assert_allclose(stable_softmax(forward_z), p)
np.testing.assert_allclose(stable_softmax(reverse_z), np.full(16, 1/16))
print("the red:", forward_pre, "->", forward_h)
print("red the:", reverse_pre, "->", reverse_h)
```

Output

```
the red: [-0.4  1.   0.5  0.3] -> [0.  1.  0.5 0.3]
red the: [ 0.1 -0.2  0.4  0.9] -> [0.1 0.  0.4 0.9]
```

8. Build the training pairs

Place a two-token window immediately before each target in `BOS BOS the red car EOS`. The helper adds the boundary tokens; pass it only the ordinary word IDs.

Context	Observed target	Context IDs	Target ID
BOS BOS	the	(1, 1)	4
BOS the	red	(1, 4)	12
the red	car	(4, 12)	7
red car	EOS	(12, 7)	2

There are four target positions and two input positions per example, giving shapes `(4, 2)` and `(4,)`. Repeating BOS fills the missing left context for this fixed-width network. EOS is a target so that stopping can be learned. These labels are observed tokens, not predictions from the untrained network.

Including the target position in its own input would reveal the answer before the model predicts it. This is target leakage. The restriction concerns positions, not token identities: when predicting the second `red` in `red red car`, the earlier `red` is valid context.

If generation chooses `bike` after `the red`, the new prefix is `the red bike` and the next two-token window is `red bike`, IDs `(12, 6)`. The reference training sentence instead supplies `red car`, IDs `(12, 7)`, to predict EOS. Training with the observed previous tokens is called teacher forcing.

In [9]

```
contexts, targets = next_token_pairs(ids)
np.testing.assert_array_equal(contexts, [[1, 1], [1, 4], [4, 12], [12, 7]])
np.testing.assert_array_equal(targets, [4, 12, 7, 2])
for context, target in zip(contexts, targets):
    print(" ".join(VOCAB[i] for i in context), "->", VOCAB[target])
repeat_contexts, repeat_targets = next_token_pairs([12, 12, 7])
np.testing.assert_array_equal(repeat_contexts[1], [BOS, 12])
assert repeat_targets[1] == 12
generated_window = [TOKEN_TO_ID[w] for w in ["red", "bike"]]
assert generated_window == [12, 6]
print("After generating bike:", generated_window)
print("Final reference training window:", contexts[-1].tolist())
```

Output

```
<bos> <bos> -> the
<bos> the -> red
the red -> car
red car -> <eos>
After generating bike: [12, 6]
Final reference training window: [12, 7]
```

9. Combine different prediction steps

The supplied probabilities come from four different prediction distributions. Multiplying along this path gives `0.5 * 0.25 * 0.8 * 0.5 = 0.05`: a **5%** probability of this exact sentence followed immediately by EOS. Omitting the last factor would give 0.1, the probability of reaching that three-word prefix without requiring the ending there. These supplied probabilities are separate from the hand-chosen model's outputs.

The total negative log-likelihood is `-ln(0.05) = 2.9957322736`. Equivalently, sum the four losses `0.6931471806 + 1.3862943611 + 0.2231435513 + 0.6931471806`. Their mean is `2.9957322736 / 4 = 0.7489330684`. All four targets count, including EOS.

The selected probabilities sum to 2.05, which is allowed. Each is chosen from a different vocabulary distribution. Alternatives at the same step must sum to one; values chosen at different steps need not. Multiplication uses conditional probabilities: for example, 0.8 is the probability of `car` given the earlier prefix. The chain rule does not assume independent tokens.

In [10]

```
selected_probabilities = np.array([0.5, 0.25, 0.8, 0.5])
sequence_probability = selected_probabilities.prod()
token_losses = -np.log(selected_probabilities)
np.testing.assert_allclose(sequence_probability, 0.05)
np.testing.assert_allclose(token_losses.sum(), -np.log(sequence_probability))
np.testing.assert_allclose(token_losses.mean(), -np.log(0.05) / 4)
print(f"Sequence probability: {sequence_probability:.2f}")
print("Token losses:", token_losses)
print(f"Total loss: {token_losses.sum():.10f}")
print(f"Mean loss: {token_losses.mean():.10f}")
```

Output

```
Sequence probability: 0.05
Token losses: [0.693147 1.386294 0.223144 0.693147]
Total loss: 2.9957322736
Mean loss: 0.7489330684
```

10. Find the window's limitation

Prefix	All IDs	IDs passed to the network
the red	(4, 12)	(4, 12)
I saw the red	(15, 14, 4, 12)	(4, 12)

The slice `ids[-2:]` keeps the final two IDs in their original order. Both calls receive identical IDs, retrieve identical embeddings, and produce identical hidden vectors, logits, and probabilities. Every output entry matches, not just the most probable token. This example gives `p(car)` about 0.306507 in both cases.

Changing the learned weights cannot recover the dropped words. For the same parameters, a deterministic network computes the same output from the same input. It could assign a different shared distribution after training, but still could not distinguish these two prefixes. The limitation is in the information it receives.

For a concrete situation, compare `At the bike shop, I picked the red ...` with `At the car dealership, I picked the red ...`. Both end with `the red`, but earlier words help a reader choose between `bike` and `car`. These longer sentences illustrate the limitation conceptually; they contain words outside the small runnable vocabulary.

In [11]

```
prefix_ids = [[TOKEN_TO_ID[w] for w in prefix.split()]
              for prefix in ["the red", "I saw the red"]]
assert prefix_ids == [[4, 12], [15, 14, 4, 12]]
short_h, short_z = context_logits(prefix_ids[0][-2:], parameters)
long_h, long_z = context_logits(prefix_ids[1][-2:], parameters)
np.testing.assert_array_equal(short_h, long_h)
np.testing.assert_array_equal(short_z, long_z)
np.testing.assert_array_equal(stable_softmax(short_z), stable_softmax(long_z))
for row in prefix_ids:
    print(row, "->", row[-2:])
print("All 16 next-token probabilities match.")
```

Output

```
[4, 12] -> [4, 12]
[15, 14, 4, 12] -> [4, 12]
All 16 next-token probabilities match.
```

11. Challenge: change the numbering

Give `the` new ID 12 and `red` new ID 4. The text `the red car` now has IDs `(12, 4, 7)`. Move each representation with its token:

Object	Required change
Embedding table E	Swap rows 4 and 12
Output weights W_out	Swap columns 4 and 12
Output bias b_out	Swap entries 4 and 12
Encoded inputs and targets	Replace old 4 with 12, and old 12 with 4
Token-to-ID and ID-to-token mappings	Apply the same exchange

The context weights do not change. Looking up the renamed IDs now retrieves the original vectors in the original order, so the hidden vector stays the same. At the output, the scores for `the` and `red` exchange slots. Softmax uses the same denominator and the probabilities exchange slots too. Reading each probability by its token name therefore gives the same answer.

Changing only the input IDs would retrieve the wrong rows: the phrase would be represented as `red the car` under the original table. Moving input rows without moving output classes would instead attach scores to the wrong names.

In this particular example all output biases are zero, so swapping them has no visible effect; it is still necessary for a general model with nonzero biases. Training targets and decoded predictions must use the new numbering as well. The executable check compares all 16 named probabilities and verifies that the target losses for the four training examples are preserved.

In [12]

```
new_to_old = np.arange(len(VOCAB))
new_to_old[[4, 12]] = new_to_old[[12, 4]]
old_to_new = np.argsort(new_to_old)
new_vocab = tuple(VOCAB[old_id] for old_id in new_to_old)
new_token_to_id = {token: i for i, token in enumerate(new_vocab)}
renamed = {key: value.copy() for key, value in parameters.items()}
renamed["E"] = parameters["E"][new_to_old]
renamed["W_out"] = parameters["W_out"][:, new_to_old]
renamed["b_out"] = parameters["b_out"][new_to_old]

new_ids = np.array([new_token_to_id[w] for w in "the red car".split()])
assert new_ids.tolist() == [12, 4, 7]
new_h, new_z = context_logits(new_ids[:2], renamed)
new_p = stable_softmax(new_z)
np.testing.assert_allclose(new_h, hidden)
for word in VOCAB:
    np.testing.assert_allclose(new_p[new_token_to_id[word]], p[TOKEN_TO_ID[word]])

_, original_training_logits = context_logits(contexts, parameters)
_, renamed_training_logits = context_logits(old_to_new[contexts], renamed)
np.testing.assert_allclose(
    token_nll(renamed_training_logits, old_to_new[targets]),
    token_nll(original_training_logits, targets),
)
print("New phrase IDs:", new_ids)
print(f"p(red) moved from slot 12 to 4: {new_p[4]:.9f}")
print("All named probabilities and training losses are preserved.")
```

Output

```
New phrase IDs: [12  4  7]
p(red) moved from slot 12 to 4: 0.112757445
All named probabilities and training losses are preserved.
```

12. Challenge: keep softmax stable

Adding the same constant to every logit does not change their differences. Algebraically, every exponential acquires the same factor `exp(1000)`, which cancels between numerator and denominator. Computing that factor explicitly would overflow in float64. Stable softmax avoids it: after subtracting each row's maximum, both the original and shifted rows have the same values (up to floating-point rounding). Each vocabulary row must sum to one.

For the two logits `(1000, -1000)`, the shifted logits are `(0, -2000)`. In real-number arithmetic the second probability is positive, approximately `exp(-2000)`. In float64 that value underflows to zero. Taking `-ln(0)` then produces positive infinity; maximum subtraction prevents large exponentials, but cannot make arbitrarily tiny probabilities representable.

Compute the loss before converting to a tiny probability: `log_normalizer - shifted_target = ln(1 + exp(-2000)) - (-2000)`. This is approximately **2000**, finite. The code's `token_nll` works this way. This example has only two classes, unlike the earlier 16-entry vocabulary.

In [13]

```
shifted_logits = logits + 1000
two_rows = np.stack([logits, shifted_logits])
two_distributions = stable_softmax(two_rows)
np.testing.assert_allclose(two_distributions[0], two_distributions[1])
np.testing.assert_allclose(two_distributions.sum(axis=-1), [1, 1])
extreme_logits = np.array([1000.0, -1000.0])
tiny_probability = stable_softmax(extreme_logits)[1]
with np.errstate(divide="ignore"):
    loss_via_probability = -np.log(tiny_probability)
direct_loss = float(token_nll(extreme_logits, 1))
assert tiny_probability == 0 and np.isposinf(loss_via_probability)
np.testing.assert_allclose(direct_loss, 2000)
print("Vocabulary row sums:", two_distributions.sum(axis=-1))
print("Rounded tiny probability:", tiny_probability)
print("Loss via probability:", loss_via_probability)
print("Loss directly from logits:", direct_loss)
```

Output

```
Vocabulary row sums: [1. 1.]
Rounded tiny probability: 0.0
Loss via probability: inf
Loss directly from logits: 2000.0
```